

Datové Struktury - otázky

Jarek Kusák

December 2025

1 Velké otázky

1.1 Definujte Splay strom. Popište, jak na něm probíhají operace Splay, Find, Insert a Delete. Popište výhody a nevýhody oproti jiným datovým strukturám, zejména vyváženým vyhledávacím stromům. Vyslovte a dokažte větu o amortizované složitosti operace Splay.

Odpověď

Definice. *Splay strom* je binární vyhledávací strom (BST), který se po každém přístupu k uzlu restrukturalizuje tak, že tento uzel přesune do kořene stromu. K restrukturalizaci se používá operace *splay*, která je složena z rotací. Splay strom neudrží žádnou explicitní vyvažovací podmínku; vyváženost je zajištěna pouze amortizovaně.

Operace $\text{Splay}(x)$

Nechť x je uzel stromu. Operace $\text{Splay}(x)$ přesouvá x do kořene pomocí následujících kroků:

- **zig:** x je synem kořene – provedeme jednu rotaci.
- **zig-zig:** x i jeho rodič jsou leví synové (nebo oba praví) – provedeme dvě rotace nad sebou.
- **zig-zag:** x je levý syn pravého syna nebo naopak – provedeme dvě rotace „do kříže“.

Tyto kroky se opakují, dokud se x nestane kořenem.

Operace FIND

Při operaci $\text{FIND}(k)$ se stromem sestupujeme jako v běžném BST. Pokud klíč k nalezeme, skončíme v uzlu x ; pokud ne, skončíme v posledním navštíveném uzlu. Tento uzel následně splayujeme do kořene.

Operace INSERT

Při INSERT(k) nejprve provedeme FIND(k).

- Pokud k již existuje, pouze splayujeme nalezený uzel.
- Pokud k neexistuje, vložíme nový list a tento uzel splayujeme.

Operace DELETE

Nejprve vyhledáme uzel s klíčem k a splayujeme jej do kořene. Po odstranění kořene rozdělíme strom na levý a pravý podstrom. Pokud pravý podstrom není prázdný, najdeme v něm minimum, splayujeme ho a připojíme levý podstrom jako jeho levého syna.

Výhody a nevýhody

Výhody.

- Velmi jednoduchá implementace (jen rotace).
- Amortizovaně $O(\log n)$ na operaci.
- Adaptivita – často používané prvky se přesouvají blíže ke kořeni.

Nevýhody.

- Neexistuje garance $O(\log n)$ v nejhorším případě pro jednu operaci.
- Časté změny struktury stromu.

Amortizovaná analýza operace Splay

Definice potenciálu. Pro uzel v označme:

$$s(v) = |T(v)|, \quad r(v) = \log s(v).$$

Potenciál stromu definujeme jako

$$\Phi = \sum_v r(v).$$

Věta (Sleator–Tarjan). Amortizovaná cena operace Splay(x) je nejvýše

$$3(r'(x) - r(x)) + 1,$$

kde $r(x)$ a $r'(x)$ jsou ranky uzlu x před a po operaci.

Důkaz. Operaci $\text{Splay}(x)$ rozložíme na jednotlivé kroky (zig, zig-zig, zig-zag). Ukáže se, že amortizovaná cena každého kroku je nejvýše $3(r_i(x) - r_{i-1}(x))$, přičemž krok zig přidá konstantu $+1$. Součtem přes všechny kroky dostaneme teleskopický součet

$$\sum_i 3(r_i(x) - r_{i-1}(x)) = 3(r'(x) - r(x)),$$

odkud plyne tvrzení věty.

Důsledek. Protože pro každý uzel platí $r(v) \leq \log n$, je amortizovaná cena $\text{Splay}(x)$ v $O(\log n)$. Totéž platí pro operace FIND, INSERT a DELETE.

1.2 Definujte (a,b)-strom. Popište, jak na něm probíhají operace Find, Insert a Delete. Popište výhody a nevýhody oproti jiným datovým strukturám, zejména vyváženým binárním vyhledávacím stromům. Rozeberte jejich složitost v nejhorším případě, včetně důkazu hloubky.

Odpověď

Definice. Necht $2 \leq a \leq b$. (a,b) -strom je zakořeněný vyhledávací strom s externími vrcholy splňující následující podmínky:

- Každý vnitřní uzel (kromě kořene) má mezi a a b syny. Typicky $b \geq 2a - 1$ (je to nastavené jako dolní limit proto, aby při štěpení nevyšli podměreční synové).
- Kořen má mezi 2 a b syny, pokud není zároveň listem.
- Všechny listy jsou ve stejné hloubce.
- Každý vnitřní uzel s k syny obsahuje $k - 1$ klíčů, které rozdělují intervaly klíčů v podstromech.

(a,b) -strom je tedy vyvážený vícecestný vyhledávací strom.

Operace FIND

Vyhledávání probíhá podobně jako v BST, ale v každém uzlu porovnáváme hledaný klíč s $k - 1$ klíči uloženými v uzlu a sestupujeme do odpovídajícího syna. Protože všechny listy jsou ve stejné hloubce, je délka hledání rovna výšce stromu.

Časová složitost je $O(\log_a n \log b)$, ale $O(\log_a n \log b) = O(\frac{\log n}{\log a} \log b) = O(\frac{\log b}{\log a} \log n) = O(\log n)$

Operace INSERT

Při vkládání klíče x :

1. Nejprve nalezneme list, kam by x patřil.
2. Klíč vložíme do listu.
3. Pokud list obsahuje více než $b - 1$ klíčů, rozdělíme jej na dva uzly s alespoň $a - 1$ klíči a prostřední klíč propagujeme do rodiče.
4. Přetečení se může šířit až ke kořeni; pokud přeteče kořen, vytvoříme nový kořen.

Časová složitost je $O(b \log_a n)$ (b kvůli překopírovávání klíčů)

Operace DELETE

Při mazání klíče x :

1. Najdeme uzel obsahující x .
2. Pokud je to vnitřní uzel, nahradíme x jeho předchůdcem nebo následníkem z listu.
3. Klíč odstraníme z listu.
4. Pokud uzel obsahuje méně než $a - 1$ klíčů (podteče), pokusíme se jej opravit:
 - buď si půjčíme klíč od sousedního bratra (pokud má bratr $> a - 1$ klíčů, pak jeho minimum převedeme do rodiče jako nový oddělovací klíč m a původní oddělovací klíč m z rodiče použijeme v našem uzlu jako nové maximum, čímž neporušíme axiomy),
 - nebo uzly sloučíme a stáhneme klíč z rodiče, čímž získáme $2a - 2$ klíčů.
5. Podtečení se může šířit až ke kořeni; pokud kořen zůstane s jedním synem, tento syn se stane novým kořenem.

Časová složitost je $O(b \log_a n)$

Výhody a nevýhody

Výhody.

- Výška stromu je logaritmická v nejhorším případě.
- Vhodné pro externí paměť (disk, cache), protože uzly mají vysoký stupeň.
- Deterministické záruky (na rozdíl od amortizovaných struktur).

Nevýhody.

- Složitější implementace než binární vyhledávací stromy.
- V paměti RAM mohou být pomalejší než jednodušší struktury kvůli větší konstantě.

Výška (a,b)-stromu

Lemma (minimální počet klíčů). Necht h je výška (a,b)-stromu a $N(h)$ minimální počet listů (listů s klíči, ne externí vrcholy!) stromu výšky h . Pak platí:

$$N(0) = 1, \quad N(h) \geq a \cdot N(h-1).$$

Odtud indukci dostaneme:

$$N(h) \geq a^h.$$

Důsledek. Je-li strom s n klíči, pak

$$a^h \leq n \Rightarrow h \leq \log_a n = O(\log n).$$

Shrnutí. (a,b)-stromy poskytují deterministické $O(\log n)$ záruky pro všechny základní operace a jsou zvláště vhodné v prostředí s hierarchií paměti.

1.3 Definujte I/O model pro správu cache a srovnajte cache-aware a cache-oblivious algoritmy. Formulujte cache-aware a cache-oblivious algoritmy pro transpozici čtvercové matice. Rozeberte jejich časovou složitost a I/O složitost.

Odpověď

I/O model (externí paměť). I/O model popisuje výpočet s hierarchií pamětí:

- rychlá paměť (cache) velikosti M ,
- pomalá paměť (hlavní paměť),
- přenos dat probíhá po blocích velikosti B .

Cena algoritmu se měří počtem *I/O operací* (přesunů bloků mezi paměťmi). Výpočet nad daty již v cache je považován za zdarma.

Ideální cache. Předpokládáme:

- optimální strategii nahrazování (ideální cache),
- plně asociativní cache,
- *tall-cache* předpoklad: $M = \Omega(B^2)$.

Cache-aware vs. cache-oblivious algoritmy

Cache-aware algoritmy. Cache-aware algoritmy:

- znají parametry M a B ,
- explicitně je využívají při návrhu (např. volba velikosti bloků).

Výhodou je optimální I/O složitost, nevýhodou je závislost na konkrétní architektuře.

Cache-oblivious algoritmy. Cache-oblivious algoritmy:

- neznají M ani B ,
- jsou navrženy bez ohledu na konkrétní paměťovou hierarchii,
- přesto dosahují asymptoticky optimální I/O složitosti (za předpokladu ideální cache).

Výhodou je přenositelnost a automatická optimalita napříč úrovněmi paměti.

Transpozice čtvercové matice

Nechť máme čtvercovou matici A velikosti $n \times n$ uloženou po řádcích a chceme spočítat A^T .

Naivní algoritmus. Pro všechna i, j provedeme přiřazení $A^T[j][i] := A[i][j]$.

- Časová složitost: $O(n^2)$.
- I/O složitost: $\Theta(n^2)$ (prakticky každý přístup způsobuje cache miss kvůli přístupům do sloupců - matice jsou v paměti typicky uložena po řádcích).

Cache-aware algoritmus (blokování)

Zvolíme bloky velikosti $k \times k$ tak, aby se oba bloky (vstupní i výstupní) vešly do cache:

$$2k^2 \leq M \quad \Rightarrow \quad k = \frac{\sqrt{M}}{2} = \Theta(\sqrt{M}).$$

Algoritmus:

1. Rozdělíme matici na bloky $k \times k$.
2. Každý blok transponujeme celý v cache.

Mimodiagonální bloky. U bloků neležících na hlavní diagonále matice je transpozice spojena nejen s otočením bloku, ale i s jeho výměnou se symetrickým blokem podél hlavní diagonály. Konkrétně pro dvojici bloků $A_{i,j}$ a $A_{j,i}$, kde $i \neq j$, je nutné:

- načíst oba vstupní bloky do cache,
- oba bloky v cache transponovat,
- a zapsat je na vzájemně prohozené pozice.

To vyžaduje současnou přítomnost dvou vstupních a dvou výstupních bloků v cache, tj. celkem $4k^2$ prvků. Podmínka na velikost bloku se tak zpřísní na

$$4k^2 \leq M,$$

což však mění pouze konstantu ve volbě k . Asymptotická volba $k = \Theta(\sqrt{M})$ i výsledná I/O složitost $O(n^2/B)$ zůstávají nezměněny.

Složitost.

- Čas: $O(n^2)$.
- I/O: Každý blok se načte a zapíše jednou:

$$O\left(\frac{n^2}{B}\right).$$

Tato složitost je asymptoticky optimální.

Cache-oblivious algoritmus (rekurzivní)

Použijeme rekurzi:

- Pokud je matice dost malá, transponujeme ji naivně.
- Jinak matici rozdělíme na čtyři kvadranty velikosti $(n/2) \times (n/2)$ a rekurzivně transponujeme:

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \Rightarrow \begin{pmatrix} A^T & C^T \\ B^T & D^T \end{pmatrix}.$$

Časová složitost. Rekurze splňuje

$$T(n) = 4T(n/2) + O(1),$$

odkud plyne $T(n) = O(n^2)$.

I/O složitost. Jakmile se podproblém vejde do cache ($n^2 \leq M$), je vyřešen bez dalších I/O. Celková I/O složitost vyjde

$$O\left(\frac{n^2}{B}\right),$$

což je optimální.

Shrnutí

- Cache-aware algoritmy využívají znalost M, B .
- Cache-oblivious algoritmy jsou nezávislé na architektuře, ale dosahují stejné asymptotické I/O složitosti.
- Rekursivní transpozice matice je klasickým příkladem cache-oblivious algoritmu s optimální I/O složitostí.

1.4 Definujte c-univerzální a k-nezávislé rodiny hešovacích funkcí. Uveďte příklady, kde nestačí c-universální rodina hešovacích funkcí, ale musíme použít k-nezávislou rodinu. Formulujte a dokažte větu o střední délce řetězce v hešování s řetězcí. Ukažte příklady c-univerzálních a k-nezávislých rodin pro hešování přirozených čísel. Pro jeden příklad dokažte c-universalitu nebo k-nezávislost, pro $k \geq 2$.

Odpověď

Definice: c-univerzalita a k-nezávislost

Nastavení. Nechť U je univerzum (typicky $[U]$), $B = [m]$ je množina příhrádek a H je rodina funkcí $h : U \rightarrow [m]$, ze které vybíráme h rovnoměrně náhodně.

Definice (c-univerzální rodina). Řekneme, že rodina H je *c-univerzální* (pro $c > 0$), jestliže pro každé dvě různé položky $x \neq y$ platí

$$\Pr_{h \in H} [h(x) = h(y)] \leq \frac{c}{m}.$$

Definice ((k,c)-nezávislá rodina). Pro celé k ($1 \leq k \leq |U|$) a $c > 0$ řekneme, že H je *(k,c)-nezávislá*, jestliže pro každou k -tici různých prvků $x_1, \dots, x_k \in U$ a každou k -tici příhrádek $a_1, \dots, a_k \in [m]$ platí

$$\Pr_{h \in H} [h(x_1) = a_1 \wedge \dots \wedge h(x_k) = a_k] \leq \frac{c}{m^k}.$$

Pokud na konstantě c nezáleží, říkáme stručně *k-nezávislá*.

Pozorování.

- Je-li H (k, c) -nezávislá pro $k > 1$, pak je i $(k-1, c)$ -nezávislá.
- Je-li H $(2, c)$ -nezávislá, pak je c -univerzální.

Kde nestačí univerzalita a je třeba k -nezávislost

Hashing s řetězci vs. otevřené adresování. Pro hešování s řetízky (chaining) typicky stačí univerzalita k řízení *střední* délky řetězce (viz věta níže). Naproti tomu u otevřeného adresování (např. lineární přidávání) a u kukačkového hešování se pro dobré garance často vyžaduje vyšší nezávislost.

Příklady z přednášky/skript.

- **Kukačkové hešování:** pro očekávaný konstantní čas INSERT při zatížení $m \geq (2 + \varepsilon)n$ se v analýze požaduje hešování z rodiny, která je nezávislá pro $k = \Theta(\log n)$ (v textu se objevuje požadavek na $\lceil 6 \log n \rceil$ -nezávislost).
- **Lineární přidávání:** existují 2-nezávislé (a dokonce i 4-nezávislé) rodiny, pro které může být očekávaný počet sond velký (v textu se uvádí, že pro alespoň jednu 2-nezávislou rodinu je to $\Omega(\sqrt{n})$ a pro alespoň jednu 4-nezávislou $\Omega(\log n)$), zatímco pro úplně náhodnou funkci při dostatečně řídké tabulce je očekávaný počet sond konstantní. Pozn. Sonda = jeden pokus podívat se do jednoho konkrétního místa tabulky.

Tedy univerzalita (resp. nízká nezávislost) může být pro otevřené adresování zásadně málo.

Věta o střední délce řetězce (chaining) + důkaz

Model (chaining). Máme m přihrádek; každá přihrádka je seznam (řetězec) prvků, které do ní hashují. Mějme množinu uložených prvků $X = \{x_1, \dots, x_n\} \subseteq U$.

Věta (o očekávaném počtu kolizí s pevnou položkou). Necht' $z \in U \setminus X$ je prvek, který v tabulce *není*. Je-li H c -univerzální rodina funkcí $U \rightarrow [m]$, potom

$$\mathbf{E}_{h \in H} [\#\{i : h(x_i) = h(z)\}] \leq \frac{cn}{m}.$$

Jinými slovy: očekávaný počet prvků v řetězci, do něhož spadne z , je nejvýše cn/m .

Důkaz. Zavedme indikátorové náhodné proměnné

$$A_i = \begin{cases} 1 & \text{pokud } h(x_i) = h(z), \\ 0 & \text{jinak.} \end{cases}$$

Pak $\mathbf{E}[A_i] = \Pr[h(x_i) = h(z)] \leq c/m$ z c -univerzality. Necht $A = \sum_{i=1}^n A_i$ je počet prvků, které kolidují se z . Linearitě očekávání dává

$$\mathbf{E}[A] = \mathbf{E}\left[\sum_{i=1}^n A_i\right] = \sum_{i=1}^n \mathbf{E}[A_i] \leq \sum_{i=1}^n \frac{c}{m} = \frac{cn}{m}.$$

Tím je důkaz hotov.

Důsledek (o složitosti chainingu). Je-li $m = \Omega(n)$, pak je očekávaná délka prohledávaného řetězce konstantní, a tedy jsou očekávané časy FIND/INSERT/DELETE konstantní (plus amortizace při občasné rehashi při změně velikosti tabulky).

Příklady rodin pro hešování přirozených čísel

Všude níže je univerzum typicky buď $[p]$ pro prvočíslo p , nebo $[2^w]$ pro strojová slova.

(1) Lineární kongruence. Pro prvočíslo p a $m \leq p$ definujeme rodinu

$$L = \{h_{a,b} \mid a, b \in [p]\}, \quad h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Tato rodina je 2-univerzální (a tedy i c -univerzální pro $c = 2$).

(2) Multiply-shift. Pro univerzum $[2^w]$ a výstup $[2^\ell]$:

$$M = \{h_a \mid a \in [2^w], a \text{ liché}\}, \quad h_a(x) = (ax)^{\langle w-\ell:w \rangle},$$

tj. vezmeme horních ℓ bitů součinu. Rodina M je 2-univerzální, ale není 2-nezávislá (např. $h_a(0) = 0$ pro všechna a). Úpravou (větší přesnost a přičtení náhodného b) dostaneme rodinu M_0 , která je 2-nezávislá.

(3) Polynomiální hešování v tělese. Necht p je prvočíslo a $k \geq 1$. Definujeme rodinu polynomiálních funkcí

$$P_k = \{h_{\mathbf{t}} \mid \mathbf{t} = (t_0, \dots, t_{k-1}) \in \mathbb{Z}_p^k\}, \quad h_{\mathbf{t}}(x) = \sum_{i=0}^{k-1} t_i x^i \in \mathbb{Z}_p.$$

Rodina P_k je $(k,1)$ -nezávislá. Chceme-li výstup do $[m]$, typicky bereme $h_{\mathbf{t}}(x) \bmod m$ při dostatečně velkém p , což zachová k -nezávislost až na konstantní faktor.

Důkaz k-nezávislosti pro $k \geq 2$ (polynomiální rodina)

Lemma. Rodina P_k je $(k,1)$ -nezávislá.

Důkaz. Vezměme libovolné *různé* body $x_1, \dots, x_k \in \mathbb{Z}_p$ a libovolné hodnoty $a_1, \dots, a_k \in \mathbb{Z}_p$. Ze standardní teorie polynomů nad tělesem plyne, že existuje právě *jeden* polynom h stupně nejvýše $k - 1$, který splňuje $h(x_i) = a_i$ pro všechna $i = 1, \dots, k$ (interpolace). Protože v P_k vybíráme koeficienty $\mathbf{t}$ rovnoměrně z $\mathbb{Z}_p^k$, je všech p^k polynomů stejně pravděpodobných a přesně jeden z nich splňuje dané k podmínek. Tedy

$$\Pr_{\mathbf{t} \in \mathbb{Z}_p^k} [h_{\mathbf{t}}(x_1) = a_1 \wedge \dots \wedge h_{\mathbf{t}}(x_k) = a_k] = \frac{1}{p^k},$$

což přesně odpovídá $(k,1)$ -nezávislosti. $\square$

1.5 Popište a analyzujte hešování s lineárním přidáváním s plně náhodnou hešovací funkcí a např. třetinovým zaplněním. Popište výhody a nevýhody oproti jiným datovým strukturám, zejména založeným na hešování.

Odpověď

Definice: otevřené adresování a lineární přidávání

Otevřené adresování. V otevřeném adresování (open addressing) má každá přihrádka (buňka pole) kapacitu nejvýše 1: prvky jsou uloženy přímo v poli délky m a hešovací funkce $h : U \rightarrow [m]$ určí *preferovanou* pozici. V případě kolize se zkouší další pozice podle *probe sequence*.

Lineární přidávání (linear probing). Fixujeme hešovací funkci $h : U \rightarrow [m]$. Sondační posloupnost pro prvek x je

$$h(x), h(x) + 1, h(x) + 2, \dots \quad (\text{indexováno modulo } m).$$

INSERT sleduje posloupnost, dokud nenajde prázdnou buňku; FIND sleduje posloupnost, dokud nenajde hledaný prvek nebo prázdnou buňku. Mazání je problematické, protože odstraněním prvku by se mohly „přerušit“ sondační posloupnosti jiných prvků; proto se používají *tombstones* a tabulka se občas přebuduje (rehash).

Proč je to netriviální: běhy (runs, clusters)

Běh. *Run* (běh, cluster) je maximální souvislý interval neprázdných buněk (indexováno modulo m). Je-li prvek uložen v běhu, pak musí být také *hešován* do nějaké buňky tohoto běhu: před během i za během je prázdná buňka, takže lineární sondování nemohlo „přeskočit“ mimo.

Vztah k počtu sond. Při hledání prvku x začínáme v buňce $h(x)$, takže počet sond je nejvýše velikost běhu, který obsahuje buňku $h(x)$. Celá analýza se tedy redukuje na odhad velikostí běhů.

Nastavení analýzy (třetinové zaplnění)

Budeme analyzovat případ:

$$m \text{ je mocnina } 2, \quad n \leq \frac{m}{3}, \quad h \text{ je plně náhodná funkce.}$$

Bez újmy na obecnosti lze předpokládat $n = m/3$, protože to maximalizuje očekávaný počet sond (a zaokrouhlování zanedbáme).

Struktura důkazu: bloky a „kritické“ bloky

Bloky. Postavíme úplný binární strom nad buňkami tabulky (indexy $0, \dots, m-1$). Uzel ve výšce t odpovídá intervalu délky 2^t , jehož začátek je zarovnán na násobek 2^t . Takové intervaly nazýváme *bloky*.

Kritický blok. Blok B velikosti 2^t je *kritický*, pokud je do něj hešováno více než

$$\frac{2}{3} \cdot 2^t$$

prvků (pozor: „hešováno do bloku“ vs. „skutečně uloženo v bloku“ je třeba rozlišovat).

Chernoffův odhad. Pro odhad pravděpodobnosti kritičnosti použijeme Chernoffovu nerovnost pro pravý ocas.

Chernoffův odhad: Nechť $X_1, \dots, X_n$ jsou nezávislé náhodné veličiny s hodnotami v $\{0, 1\}$ a označme

$$X = \sum_{i=1}^n X_i, \quad \mu = \mathbb{E}[X].$$

Potom pro libovolné $c > 1$ platí

$$\mathbb{P}(X > c\mu) \leq \left(\frac{e^{c-1}}{c^c} \right)^\mu.$$

Lemma (pravděpodobnost kritického bloku). Nechť B je blok velikosti 2^t . Pak pravděpodobnost, že B je kritický, je nejvýše

$$q^{2^t}, \quad \text{kde } q = (e/4)^{1/3} \approx 0.879.$$

Důkaz. Pro každý prvek x_i definujeme indikátor $X_i = 1$ právě tehdy, když $h(x_i) \in B$. Pak $\mathbf{E}[X_i] = 2^t/m$ a

$$X = \sum_{i=1}^n X_i$$

počítá počet prvků hešovaných do B . Máme

$$\mu = \mathbf{E}[X] = n \cdot \frac{2^t}{m} = \frac{m}{3} \cdot \frac{2^t}{m} = \frac{2^t}{3}.$$

Kritičnost znamená $X > \frac{2}{3}2^t = 2\mu$. Chernoff s $c = 2$ dává

$$\Pr[X \geq 2\mu] \leq (e/4)^\mu = (e/4)^{2^t/3} = q^{2^t}.$$

□

Dlouhý běh implikuje kritický blok

Lemma (běh $\Rightarrow$ kritický blok). Necht R je běh délky alespoň $2^{\ell+2}$ a B_0, B_1, B_2, B_3 jsou první čtyři bloky velikosti 2^ℓ , které R protíná. Pak alespoň jeden z těchto bloků je kritický.

Intuice. Dlouhý běh obsahuje mnoho obsazených buněk; protože před příslušným intervalem je prázdná buňka, všechny tyto uložené prvky musely být hešovány „dovnitř“. Pokud by žádný z relevantních bloků nebyl kritický, kapacita „hešů do bloků“ by nestačila vysvětlit tolik obsazených buněk.

Pravděpodobnost velkého běhu kolem $h(x)$

Lemma. Necht R je běh obsahující buňku $h(x)$ a $|R| \in [2^{\ell+2}, 2^{\ell+3})$. Pak alespoň jeden z 12 bloků velikosti 2^ℓ je kritický: blok obsahující $h(x)$, 8 bloků před ním a 3 bloky za ním.

Důsledek (odhad pravděpodobnosti). Použijeme sjednocovací odhad (union bound) a lemma o kritickém bloku:

$$\Pr[|R| \in [2^{\ell+2}, 2^{\ell+3})] \leq 12 \cdot q^{2^\ell}.$$

Očekávaná délka běhu je konstanta

Pokryjeme všechny možné délky běhu intervaly $[2^{\ell+2}, 2^{\ell+3})$ a výjimečným intervalem $[0, 3]$. Nahradíme každou délku horní mezí svého intervalu (tím očekávání jen vzroste):

$$\mathbf{E}[|R|] \leq 3 \cdot \Pr[|R| \leq 3] + \sum_{\ell \geq 0} 2^{\ell+3} \cdot \Pr[|R| \in [2^{\ell+2}, 2^{\ell+3})].$$

První člen omezíme 3. Druhý odhadneme předchozím důsledkem:

$$\mathbf{E}[|R|] \leq 3 + \sum_{\ell \geq 0} 2^{\ell+3} \cdot 12q^{2^\ell} = 3 + 96 \sum_{\ell \geq 0} 2^\ell q^{2^\ell}.$$

Protože $0 < q < 1$, řada konverguje (ve skriptech se to převádí na řadu tvaru $\sum iq^i$), a tedy $\mathbf{E}[|R|] = O(1)$, s konstantou nezávislou na n, m .

Věta (konstantní očekávaný počet sond). Za předpokladů m mocnina 2, $n \leq m/3$ a h plně náhodná hešovací funkce je očekávaný počet sond při hledání položky x omezen konstantou nezávislou na n, m .

Výhody a nevýhody (vs. jiné hešování a struktury)

Výhody.

- **Cache-friendliness:** přístup je sekvenční (lineární sken), což dobře sedí na cache.
- **Jednoduchost:** operace FIND/INSERT jsou jednoduché a bez pointerů.
- **Při silném hashování a řídké tabulce** (např. konstantní zaplnění) má očekávaně konstantní počet sond.

Nevýhody.

- **Clustering (běhy):** lineární přidávání tvoří shluky a bez „silného“ hashování se může dramaticky zhoršit.
- **Mazání:** vyžaduje tombstones a občasné přebudování tabulky.
- **Citlivost na (ne)nezávislost:** skripta uvádějí, že pro některé nízko-nezávislé rodiny mohou být očekávané počty sond velké (např. $\Omega(\sqrt{n})$ pro některé 2-nezávislé rodiny, $\Omega(\log n)$ pro některé 4-nezávislé atd.), zatímco pro plně náhodnou funkci je to při řídké tabulce konstantní.

Srovnání s chainingem a kukačkou.

- **Chaining** je méně citlivý na shluky a pro očekávanou délku řetězce typicky stačí univerzalita, ale potřebuje pointer a hůře se chová v cache.
- **Kukačkové hešování** dává worst-case $O(1)$ pro lookup, ale insert může vyžadovat rehash; analýza navíc typicky vyžaduje vyšší nezávislost hešů.

**1.6 Definujte vícerozměrné intervalové stromy (range trees).
Rozeberte prostorovou složitost datové struktury a časovou složitost konstrukce a obdélníkových dotazů (bonus: včetně zrychlení kaskádováním).**

Zadání problému (range searching)

Máme množinu bodů

$$P = \{(x_1, \dots, x_d)\} \subseteq \mathbb{R}^d$$

a chceme odpovídat na dotazy typu:

$$[x_1^-, x_1^+] \times [x_2^-, x_2^+] \times \dots \times [x_d^-, x_d^+],$$

tj. vypsát (nebo spočítat) všechny body ležící v daném osově zarovnaném d -rozměrném obdélníku.

Jednorozměrný případ (základní stavební kámen)

V 1D ukládáme body do vyváženého binárního vyhledávacího stromu podle souřadnice x . Každý uzel v odpovídá intervalu $\text{int}(v)$ na ose x .

Intuice dotazu. Chceme vypsát všechny body s $x \in Q$. Stromem sestupujeme tak, že:

- pokud $\text{int}(v) \cap Q = \emptyset$, podstrom ignorujeme,
- pokud $\text{int}(v) \subseteq Q$, vypíšeme celý podstrom,
- jinak pokračujeme rekurzivně do obou synů.

Algoritmus RANGEQUERY v 1D (podle přednášky).

RangeQuery(v , Q):

1. pokud v je externí: návrat
2. pokud $\text{int}(v) \cap Q = \emptyset$: návrat
3. pokud $\text{int}(v) \subseteq Q$: vypiš celý $T(v)$; návrat
4. $Q_L := Q \cap \text{int}(\text{left}(v))$
 $Q_R := Q \cap \text{int}(\text{right}(v))$
5. pokud $Q_L \neq \emptyset$: RangeQuery($\text{left}(v)$, Q_L)
6. pokud $Q_R \neq \emptyset$: RangeQuery($\text{right}(v)$, Q_R)

Složitost. Algoritmus navštíví:

- $O(\log n)$ uzlů na hranách dotazu,
- a $O(k)$ uzlů odpovídajících vypsáním bodům.

Celková složitost je tedy

$$O(\log n + k).$$

Definice 2D intervalového stromu (range tree)

Primární strom. Nejprve sestrojíme vyvážený vyhledávací strom podle souřadnice x . Každý uzel v reprezentuje interval $\text{int}(v)$ na ose x a množinu bodů

$$P_v = \{p \in P \mid x(p) \in \text{int}(v)\}.$$

Sekundární struktury. Ke každému uzlu v připojíme sekundární strukturu: vyvážený vyhledávací strom (nebo setříděné pole) nad množinou P_v , seřazenou podle souřadnice y .

Intuitivně: každý uzel „zná všechny body pod sebou, uspořádané podle y “.

Dotaz v 2D

Chceme odpovědět na dotaz

$$[x^-, x^+] \times [y^-, y^+].$$

Postup.

1. V primárním stromu podle x nalezneme $O(\log n)$ rozkladných uzlů, jejichž intervaly přesně pokrývají $[x^-, x^+]$.
2. Pro každý takový uzel v provedeme v jeho sekundární struktuře jedno-rozměrný dotaz RANGEQUERY na interval $[y^-, y^+]$.

Intuice. Rozkladné uzly pokrývají dotaz v ose x bez překryvů. V každém z nich už řešíme jen „jednorozměrný problém podle y “.

Složitost dotazu (bez kaskádování).

$$O(\log n) \cdot O(\log n + k_v) = O(\log^2 n + k),$$

kde k je celkový počet nalezených bodů.

Prostorová a konstrukční složitost (2D)

Prostor. Každý bod je uložen v sekundární struktuře každého uzlu na cestě od kořene k listu primárního stromu:

$$O(n \log n).$$

Konstrukce. Primární strom sestrojíme v $O(n \log n)$. Sekundární struktury:

- naivně: $O(n \log^2 n)$,
- efektivně sdílením setříděných seznamů: $O(n \log n)$.

Zrychlení: frakční kaskádování

Myšlenka. Sekundární struktury v sousedních uzlech jsou silně příbuzné. Frakční kaskádování umožní:

- jediné binární vyhledávání,
- následné přechody v konstantním čase.

Složitost dotazu s kaskádováním.

$$O(\log n + k).$$

Zobecnění do d rozměrů

Definice. d -rozměrný intervalový strom definujeme rekurzivně:

- primární strom podle první souřadnice,
- v každém uzlu $(d-1)$ -rozměrný intervalový strom.

Složitosti.

$$\text{Prostor} = O(n \log^{d-1} n), \quad \text{Dotaz} = \begin{cases} O(\log^d n + k) & \text{bez kaskádování,} \\ O(\log^{d-1} n + k) & \text{s kaskádováním.} \end{cases}$$

Shrnutí

- Range trees řeší ortogonální rozsahové dotazy.
- Základem je 1D RANGEQUERY s časem $O(\log n + k)$.
- Ve vyšších dimenzích se platí faktorem $\log n$ za každou dimenzi.
- Frakční kaskádování zlepšuje dotazy o jeden logaritmus.

1.7 Definujte suffixové pole a LCP pole. Popište a analyzujte algoritmy na jejich konstrukci (pro suffixové pole stačí ve skoro lineárním čase). Popište příklad úlohy, kterou umí tato pole efektivně řešit.

Odpověď

Definice suffixového pole

Suffixy. Nechť $S = s_1 s_2 \cdots s_n$ je řetězec nad abecedou Σ . Pro $1 \leq i \leq n$ definujeme *suffix*

$$S_i = s_i s_{i+1} \cdots s_n.$$

Definice (suffixové pole). *Suffixové pole* (suffix array, SA) je pole $SA[1..n]$, které obsahuje permutaci indexů $\{1, \dots, n\}$ takovou, že

$$S_{SA[1]} < S_{SA[2]} < \dots < S_{SA[n]}$$

v lexikografickém pořadí.

Jinými slovy, $SA[i]$ je počáteční pozice i -tého nejmenšího sufixu.

Intuitivně. SA říká: na které pozici/indexu v seně (řetězci) začíná i -tý suffix v lexikografickém pořadí.

Definice LCP pole

Definice (LCP). *LCP pole* je pole $LCP[1..n]$ definované jako:

$$LCP[1] = 0, \quad LCP[i] = \text{lcp}(S_{SA[i-1]}, S_{SA[i]}) \quad \text{pro } i \geq 2,$$

kde $\text{lcp}(A, B)$ je délka nejdelšího společného prefixu řetězců A, B .

Naivní konstrukce suffixového pole

1. Vygenerujeme všech n sufixů.
2. Setřídíme je lexikograficky.

Složitost.

- Počet sufixů: n .
- Porovnání sufixů může trvat až $O(n)$.

Celková složitost je $O(n^2 \log n)$, což je příliš pomalé.

Konstrukce v $O(n \log n)$: zdvojování

Myšlenka. Suffixy třídíme podle prefixů délky 2^k . Pro $k = 0$ třídíme podle jednotlivých znaků, pro $k + 1$ třídíme podle dvojic tříd (c_i, c_{i+2^k}) .

Algoritmus.

1. Inicializace: přiřadíme každému znaku třídu.
2. Pro $k = 0, 1, 2, \dots$:
 - setřídíme suffixy podle dvojic tříd délky 2^k ,
 - vytvoříme nové třídy ekvivalence.
3. Skončíme, jakmile $2^k \geq n$.

Složitost.

$$O(n \log n),$$

protože máme $O(\log n)$ iterací a každá lze provést v lineárním čase (radix sort).

Konstrukce ve skoro lineárním čase

Skripta uvádějí existenci algoritmů:

- SA-IS,
- DC3 / skew algoritmus,

které sestrojí suffixové pole v čase $O(n)$ (resp. $O(n\alpha(n))$) za předpokladu konstantní abecedy. Pro účely této otázky postačuje znalost základní myšlenky a výsledné časové složitosti algoritmu.

Konstrukce ve skoro lineárním čase: DC3 (skew algoritmus)

Myšlenka. Skew algoritmus (DC3) sestrojuje suffixové pole v lineárním čase pomocí techniky *rozděl a panuj*. Suffixy se rozdělí podle počáteční pozice modulo 3 na tři skupiny.

Postup (intuice).

- Rekurzivně se setřídí pouze suffixy na pozicích $i \equiv 1, 2 \pmod{3}$ (tj. 2/3 všech sufixů), které lze porovnávat pomocí trojic znaků.
- Získané pořadí se použije k přiřazení čísel (ranků).
- Suffixy na pozicích $i \equiv 0 \pmod{3}$ se poté setřídí lineárně sléváním s již setříděnými suffixy.

Složitost. Algoritmus splňuje rekurenci

$$T(n) = T(2n/3) + O(n),$$

a proto sestrojí suffixové pole v čase $O(n)$ (případně $O(n\alpha(n))$ v závislosti na modelu).

Konstrukce LCP pole

Kasaiův algoritmus. Necht' $\text{rank}[i]$ je pozice suffixu S_i v suffixovém poli. Kasaiův algoritmus postupně prochází suffixy v textovém pořadí a udržuje délku aktuálního LCP.

Algoritmus.

1. Spočítáme pole rank.
2. Nastavíme $k = 0$.
3. Pro $i = 1, \dots, n$:
 - pokud $\text{rank}[i] = 1$, pak $k = 0$,
 - jinak porovnáváme S_i a jeho lexikografického následníka znak po znaku, začínaje od k ,
 - uložíme výsledek do LCP a snížíme k o 1.

Složitost. Celkový počet porovnání znaků je $O(n)$, tedy

$$\text{čas} = O(n), \quad \text{paměť} = O(n).$$

Použití: nejdelší společný podřetězec

Úloha. Máme dva řetězce A a B a chceme najít jejich nejdelší společný podřetězec.

Řešení pomocí SA a LCP.

1. Sestavíme řetězec $S = A\#B\$,$ kde $\#, \$$ jsou speciální znaky.
2. Spočítáme SA a LCP pole pro S .
3. Projdeme pole LCP a hledáme maximum $\text{LCP}[i]$ takové, že sufixy $\text{SA}[i]$ a $\text{SA}[i - 1]$ pocházejí z různých řetězců (mají mezi sebou křížek).

Správnost. Každý společný podřetězec odpovídá společnému prefixu dvou sufixů, a nejdelší z nich se nutně objeví u dvou lexikograficky sousedních sufixů.

Složitost.

$$O(n),$$

kde $n = |A| + |B|$.

Shrnutí

- Suffixové pole je kompaktní náhrada suffixového stromu.
- Lze je sestavit v $O(n \log n)$ i v lineárním čase.
- LCP pole umožňuje efektivní řešení mnoha řetězcových úloh.

2 Malé otázky

2.1 Popište dynamické pole, tedy „nafukovací pole“ se zvětšováním a zmenšováním. Analyzujte jeho amortizovanou složitost.

Odpověď

Definice. Dynamické pole je sekvenční datová struktura s kapacitou C a počtem uložených prvků $n \leq C$. Prvky jsou uloženy v souvislé paměti a podporují přístup v čase $O(1)$.

Zvětšování a zmenšování.

- APPEND: pokud $n < C$, vložení stojí $O(1)$; pokud $n = C$, pole se realokuje na kapacitu $2C$ a všech n prvků se překopíruje.
- POP: po odebrání prvku, pokud $n < C/4$, zmenšíme kapacitu na $C/2$, aby se zabránilo oscilacím.

Agregační amortizovaná analýza. Uvažujme posloupnost m operací APPEND a POP. Reálná cena běžné operace bez realokace je $O(1)$. Realokace při zvětšení nebo zmenšení kapacity stojí $\Theta(C)$, kde C je kapacita pole těsně před realokací.

Mezi dvěma po sobě jdoucími realokacemi musí proběhnout alespoň $\Theta(C)$ běžných operací:

- mezi dvěma zvětšeními je třeba vložit alespoň $C/2$ prvků,
- mezi zmenšením a dalším zmenšením nebo zvětšením je třeba provést alespoň $C/4$ operací.

Celková cena všech realokací v celé posloupnosti je tedy $O(n)$.

Závěr. Součet reálných nákladů n operací APPEND/POP je $O(m)$, takže každá z těchto operací má amortizovanou složitost $O(1)$, zatímco přístup k prvkům zůstává $O(1)$ v nejhorším případě.

2.2 Popište vyhledávací stromy s líným vyvažováním (BB[α]-stromy). Analyzujte jejich amortizovanou složitost. Uveďte příklad jejich použití.

Odpověď

Definice. BB[α]-strom je binární vyhledávací strom, kde pro každý uzel v platí

$$\max\{|T_L(v)|, |T_R(v)|\} \leq \alpha \cdot |T(v)|, \quad \frac{1}{2} < \alpha < 1.$$

Podmínka zajišťuje váhovou vyváženost, ale strom se vyvažuje *líně*.

Líné vyvažování. Po operaci INSERT nebo DELETE se kontrolují uzly na cestě ke kořeni. Pokud je v uzlu v porušena vyvažovací podmínka, celý podstrom $T(v)$ se přestaví na dokonale vyvážený binární strom (vytvořený např. z inorder průchodu).

Výška. Z podmínky vyváženosti plyne, že na každé cestě od kořene k listu se velikost podstromu zmenšuje alespoň o konstantní faktor α , tedy výška stromu je $O(\log n)$.

Amortizovaná analýza. Pro uzel v označme $s(v) = |T(v)|$, $s(\ell(v))$ a $s(r(v))$ velikosti levého a pravého podstromu. Definujeme potenciál uzlu

$$\varphi(v) := |s(\ell(v)) - s(r(v))|,$$

a celkový potenciál stromu

$$\Phi := \sum_v \varphi(v).$$

Insert bez vyvážení. Při vložení nového listu se změní velikosti pouze uzlů na cestě ke kořeni, jejíž délka je $O(\log n)$. Pro každý takový uzel se hodnota $\varphi(v)$ změní nejvýše o konstantu, tedy

$$\Delta\Phi \in O(\log n).$$

Reálná cena operace je $O(\log n)$, a tedy i amortizovaná cena je $O(\log n)$.

Vyvážení v uzlu v . Vyvážení (přestavba) podstromu $T(v)$ nastane, pokud je porušena podmínka

$$s(\ell(v)) > \alpha s(v) \quad \text{nebo} \quad s(r(v)) > \alpha s(v).$$

Bez újmy na obecnosti uvažujme $s(\ell(v)) > \alpha s(v)$. Pak platí

$$\varphi(v) = s(\ell(v)) - s(r(v)) \geq (2\alpha - 1) s(v) = \Omega(s(v)),$$

protože $\alpha > \frac{1}{2}$.

Po přestavbě je $T(v)$ dokonale vyvážený, tedy $\varphi(u) = 0$ pro všechna $u \in T(v)$, a celkový potenciál se sníží o $\Omega(s(v))$.

Poznámka k definici potenciálu. Aby amortizovaná analýza fungovala, definujeme potenciál uzlu v jako

$$\varphi(v) = \begin{cases} 0 & \text{pokud } |s(\ell(v)) - s(r(v))| \leq 1, \\ |s(\ell(v)) - s(r(v))| & \text{jinak.} \end{cases}$$

Tímto „ořezáním“ zanedbáváme malé rozdíly velikostí podstromů. Po přestavbě podstromu $T(v)$ jsou všechny jeho uzly dokonale vyvážené, tedy mají potenciál 0, což zaručuje pokles celkového potenciálu o $\Omega(s(v))$ a umožňuje zaplatit cenu přestavby amortizovaně.

Závěr. Reálná cena přestavby je $\Theta(s(v))$, ale pokles potenciálu je $\Omega(s(v))$, takže amortizovaná cena vyvážení je $O(1)$. Celkově tedy platí:

INSERT, DELETE $\in O(\log n)$ amortizovaně.

Použití. BB[α]-stromy jsou vhodné jako jednodušší alternativa k AVL nebo červeno-černým stromům, zejména tam, kde nevádí občasná drahá operace, ale je důležitá logaritmická složitost v součtu operací.

2.3 Navrhněte operace Find, Insert a Delete na Splay stromu. Analyzujte jejich amortizovanou složitost. Větu o složitosti operace Splay stačí vyslovit.

Odpověď

Operace Find. Vyhledávání probíhá jako v běžném binárním vyhledávacím stromu. Po nalezení hledaného klíče (nebo po zastavení hledání) se provede operace SPLAY na posledním navštíveném uzlu, která jej přesune do kořene stromu.

Operace Insert. Nejprve provedeme FIND.

- Pokud klíč již existuje, strom se pouze splayuje.
- Pokud neexistuje, vloží se nový list na odpovídající místo a tento uzel se následně splayuje do kořene.

Operace Delete. Vyhledáme uzel s daným klíčem a splayujeme jej do kořene. Po odstranění kořene spojíme levý a pravý podstrom: například tak, že splayujeme minimum pravého podstromu a připojíme k němu levý podstrom.

Věta (Sleator–Tarjan). Amortizovaná cena operace SPLAY je $O(\log n)$.

Důsledek. Protože každá základní operace obsahuje nejvýše jeden splay, platí:

$$\text{FIND, INSERT, DELETE} \in O(\log n) \text{ amortizovaně.}$$

2.4 Vyslovte a dokažte větu o amortizovaném počtu změn uzlů pro operace Insert a Delete v $(a, 2a)$ -stromech. Jak se to liší pro $(a, 2a-1)$ -stromy?

Odpověď

Věta (jen inserty). Posloupnost m operací INSERT v (a, b) -stromu (na začátku prázdném) změní celkem $O(m)$ vrcholů (tj. proběhne jen $O(m)$ splitů).

Důkaz. Každý split vytvoří nový vrchol (přibude alespoň jeden nový list/uzel). Po m vkládáních je v stromu nejvýše m prvků, a tedy i nejvýše $O(m)$ vrcholů. Proto počet splitů (a tedy počet strukturálních změn) je $O(m)$. $\square$

Věta (inserty i deletey). Posloupnost m operací INSERT a DELETE na začátku prázdném $(a, 2a)$ -stromu změní celkem $O(m)$ vrcholů (amortizovaně $O(1)$ strukturálních změn na operaci).

Důkaz (potenciál jako v Marešovi). Uvažujme $(a, 2a)$ -strom, kde (mimo kořen) má každý uzel mezi $a - 1$ a $2a - 1$ klíči. Při operaci se může dočasně objevit uzel s $2a$ klíči (přetečení po insertu) nebo s $a - 2$ klíči (podtečení po delete).

Definujeme *příspěvek uzlu* podle počtu klíčů:

$$f(k) := \text{příspěvek vrcholu s } k \text{ klíči} \quad (k \in \{a - 2, a - 1, \dots, 2a\}),$$

a potenciál

$$\Phi := \sum_{v \text{ vnitřní, } v \neq \text{kořen}} f(k(v)),$$

kde $k(v)$ je počet klíčů v uzlu v .

Chceme, aby platilo:

1. **Lokální změna je malá:** při běžném insertu/deletu bez strukturální změny se změní $k(v)$ jen o ± 1 na $O(\log n)$ uzlech na cestě, a zvolíme f tak, aby

$$|f(k \pm 1) - f(k)| \leq c$$

pro nějakou konstantu c (tedy $\Delta\Phi = O(\log n)$, což odpovídá ceně sestupu).

2. **Split se zaplatí poklesem potenciálu:** při přetečení vznikne uzel s $2a$ klíči a split jej nahradí dvěma uzly s a a $a - 1$ klíči. Požadujeme

$$f(2a) \geq f(a) + f(a - 1) + c + 1.$$

Pak amortizovaná cena splitu (reálná 1 plus $\Delta\Phi$) je ≤ 0 .

3. **Merge se zaplatí poklesem potenciálu:** při podtečení se může objevit uzel s $a - 2$ klíči a opravujeme jej sloučením se sousedem (typicky s uzlem s $a - 1$ klíči) za vzniku uzlu s $2a - 2$ klíči. Požadujeme

$$f(a - 2) + f(a - 1) \geq f(2a - 2) + c + 1,$$

a opět amortizovaná cena merge vyjde ≤ 0 .

Konkrétní volba f (jako na tabuli). Stačí např. zvolit:

k	$a-2$	$a-1$	a	$\dots$	$2a-2$	$2a-1$	$2a$
$f(k)$	2	1	0	$\dots$	0	2	4

(přičemž $f(k) = 0$ pro $a \leq k \leq 2a - 2$). Pak lze vzít konstantu $c = 2$ a snadno ověřit výše uvedené nerovnosti (2) a (3): split i merge snižují potenciál alespoň o konstantu větší než jejich reálná cena.

Závěr. Každý split/merge má *nekladnou amortizovanou cenu*, a tedy počet strukturálních změn v libovolné posloupnosti m operací je $O(m)$. $\square$

Rozdíl pro (a,2a-1)-stromy. U (a,2a-1)-stromů je „rezerva“ o 1 menší: po splitu/merge vznikají uzly s hrančním počtem klíčů častěji, a nelze (v tomto jednoduchém schématu) navrhnout takové f , aby současně platily analogické nerovnosti pro všechny lokální opravy. Výsledkem je, že počet strukturálních změn na operaci může být v nejhorším případě až $\Theta(\log n)$ (změny se mohou šířit až ke kořeni).

2.5 Analyzujte k-cestný Mergesort v cache-aware modelu. Jaká je optimální volba k ?

Odpověď

I/O model (cache-aware). Použijeme dvouúrovňový I/O model se stránkami (bloky) velikosti B a cache velikosti M . Cena algoritmu je počet přenosů bloků mezi pamětí a cache (I/O operací).

k -cestný Mergesort. Pole velikosti n rozdělíme na k stejně velkých částí, každou rekurzivně setřídíme a poté je sloučíme k -cestným merge.

Podmínka na slučování (buffering). Aby byl merge I/O-efektivní, držíme v cache:

- pro každý ze k vstupů buffer velikosti B (čteme po blocích),
- jeden výstupní buffer velikosti B (zapisujeme po blocích).

Proto potřebujeme

$$(k + 1)B \leq M.$$

Za této podmínky je samotné k -cestné slučování *lineární v I/O*: přečteme a zapíšeme celkem $\Theta(n/B)$ bloků, tedy

$$\text{merge}(n) = \Theta\left(\frac{n}{B}\right).$$

Rekurence pro I/O složitost. Pro $n \leq M$ lze setřídít v cache bez dalších I/O nad rámec načtení/zápisu, tj. báze. Pro $n > M$ platí rekurence

$$T(n) = kT(n/k) + \Theta\left(\frac{n}{B}\right), \quad T(n) = \Theta\left(\frac{n}{B}\right) \text{ pro } n \leq M.$$

Hloubka rekurze je počet úrovní, než se velikost problému zmenší na M :

$$\ell = \log_k \frac{n}{M}.$$

Na každé úrovni zaplatíme $\Theta(n/B)$ (slučování všech běhů dohromady), takže

$$T(n) = \Theta\left(\frac{n}{B} \cdot \log_k \frac{n}{M}\right).$$

Optimální volba k . Chceme maximalizovat k (zmenšit počet úrovní), ale musíme dodržet $(k + 1)B \leq M$, tj.

$$k = \Theta\left(\frac{M}{B}\right).$$

Po dosazení dostaneme

$$T(n) = \Theta\left(\frac{n}{B} \cdot \log_{M/B} \frac{n}{M}\right) = \Theta\left(\frac{n}{B} \cdot \log_{M/B} \frac{n}{B}\right),$$

kde poslední tvar je standardní (liší se jen o konstantní posun v logaritmu).

Poznámka (cache-oblivious). Toto je *cache-aware* algoritmus, protože volba k používá parametry M, B . Existují i *cache-oblivious* varianty mergesortu (např. Funnelsort), které dosahují stejné asymptotické I/O složitosti bez znalosti M, B , ale tato otázka se ptá konkrétně na cache-aware analýzu.

2.6 Vyslovte a dokažte Sleatorovu–Tarjanovu větu o kompetitivě LRU. Popište přínos této věty pro analýzu cache-oblivious algoritmů.

Odpověď

Model. Uvažujeme algoritmy správy paměti, které obsluhují stejnou posloupnost přístupů $x_1, x_2, \dots, x_n$:

- LRU s cache velikostí M_{LRU} ,
- OPT, optimální offline algoritmus, s cache velikostí M_{OPT} ,

kde platí $M_{\text{LRU}} > M_{\text{OPT}} \geq 1$. Cena algoritmu je počet přístupů, při nichž požadovaná položka není v cache a musí být načtena z pomalé paměti.

Věta (Sleator–Tarjan). Pro libovolnou přístupovou posloupnost σ platí

$$\text{LRU}(\sigma) \leq \frac{M_{\text{LRU}}}{M_{\text{LRU}} - M_{\text{OPT}}} \cdot \text{OPT}(\sigma) + M_{\text{OPT}}.$$

Poznámka. Podmínka $M_{\text{LRU}} > M_{\text{OPT}}$ je nutná: pokud by obě cache měly stejnou velikost, existují přístupové posloupnosti, na nichž se LRU může chovat libovolně hůře než OPT.

Důkaz (intuice pomocí epoch). Rozdělíme přístupovou posloupnost $x_1, \dots, x_n$ na po sobě jdoucí úseky zvané *epochy* $E_0, E_1, \dots, E_k$ následovně:

- Epocha E_0 začíná na začátku posloupnosti.
- Každá další epocha končí v okamžiku, kdy LRU obsloužila přístupy na přesně M_{LRU} různých položek od začátku této epochy.

Z definice epoch plyne:

- v epoše E_0 zaplatí LRU nejvýše M_{LRU} ,
- v každé další epoše E_i (pro $i > 0$) zaplatí LRU *přesně* M_{LRU} , protože se v ní objeví M_{LRU} různých položek.

Nyní uvažujme libovolnou epochu E_i pro $i > 0$ a podívejme se, co se v ní musí stát z pohledu algoritmu OPT. Rozlišíme dva případy:

1. **Všechny položky v epoše jsou nové.** Pak se v epoše objevilo alespoň M_{LRU} různých položek. Protože OPT má cache velikosti pouze M_{OPT} , musí v této epoše zaplatit alespoň $M_{\text{LRU}} - M_{\text{OPT}}$ načtení.
2. **Některá položka se v epoše opakuje.** Pak LRU tuto položku mezi dvěma přístupy zapomněla, což znamená, že mezitím musela z cache vytlačit alespoň M_{LRU} jiných položek. I v tomto případě musí OPT, který má menší cache, zaplatit alespoň jedno načtení v dané epoše.

V obou případech tedy platí, že každá epocha (kromě první) přispěje alespoň konstantním zlomkem ceny algoritmu OPT, zatímco LRU v ní zaplatí přesně M_{LRU} . Sečtením přes všechny epochy dostaneme, že celkový počet načtení algoritmu LRU je omezen uvedeným výrazem. $\square$

Přínos pro cache-oblivious algoritmy. Cache-oblivious algoritmy se analyzují za předpokladu optimální správy paměti (OPT). Sleatorova–Tarjanova věta zaručuje, že skutečná implementace s politikou LRU a o něco větší cache má počet I/O operací jen o konstantní faktor větší. To ospravedlňuje používání cache-oblivious analýzy pro reálné architektury.

2.7 Popište systém hešovacích funkcí odvozený ze skalárního součinu. Dokažte, že je to 1-univerzální systém ze $\mathbb{Z}_p^k$ do $\mathbb{Z}_p$.

Odpověď

Definice systému. Nechť p je prvočíslo a uvažujme univerzum $\mathbb{Z}_p^k$. Definujeme rodinu hešovacích funkcí

$$H = \{h_{\mathbf{a}} \mid \mathbf{a} \in \mathbb{Z}_p^k\}, \quad h_{\mathbf{a}}(\mathbf{x}) = \langle \mathbf{a}, \mathbf{x} \rangle = \sum_{i=1}^k a_i x_i \pmod{p}.$$

Tvrzení. Rodina H je 1-univerzální, tj. pro libovolné různé $\mathbf{x} \neq \mathbf{y} \in \mathbb{Z}_p^k$ platí

$$\Pr_{\mathbf{a}}[h_{\mathbf{a}}(\mathbf{x}) = h_{\mathbf{a}}(\mathbf{y})] = \frac{1}{p}.$$

Důkaz. Podmínka $h_{\mathbf{a}}(\mathbf{x}) = h_{\mathbf{a}}(\mathbf{y})$ je ekvivalentní

$$\langle \mathbf{a}, \mathbf{x} - \mathbf{y} \rangle = 0 \pmod{p}.$$

Protože $\mathbf{x} \neq \mathbf{y}$, existuje alespoň jedna souřadnice, ve které se liší, tedy vektor $\mathbf{d} = \mathbf{x} - \mathbf{y}$ je nenulový. Lineární rovnice $\langle \mathbf{a}, \mathbf{d} \rangle = 0$ má přesně p^{k-1} řešení v $\mathbb{Z}_p^k$. Proto

$$\Pr[h_{\mathbf{a}}(\mathbf{x}) = h_{\mathbf{a}}(\mathbf{y})] = \frac{p^{k-1}}{p^k} = \frac{1}{p}.$$

$\square$

—

2.8 Popište systém hešovacích funkcí založených na lineární kongruenci. Dokažte, že je to 2-nezávislý systém ze $\mathbb{Z}_p$ do $[m]$ (můžete využít lemma o modulení).

Odpověď

Definice systému. Necht p je prvočíslo a $m \leq p$. Definujeme rodinu

$$H = \{h_{a,b} \mid a, b \in \mathbb{Z}_p\}, \quad h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Lemma o modulení (bez důkazu). Necht X je rovnoměrné v $\mathbb{Z}_p$. Pak $X \bmod m$ je téměř rovnoměrné v $[m]$, tj. každá hodnota má pravděpodobnost nejvýše $\frac{2}{m}$.

Tvrzení. Rodina H je 2-nezávislá (a tedy $O(1)$ -univerzální).

Důkaz (náčrt dle skript). Vezměme různé $x \neq y \in \mathbb{Z}_p$. Zobrazení

$$(a, b) \mapsto (ax + b, ay + b) \pmod{p}$$

je bijekce na $\mathbb{Z}_p^2$. Proto jsou dvojice $(h(x), h(y))$ téměř rovnoměrné v $[m]^2$. Z lemma o modulení plyne

$$\Pr[h(x) = u \wedge h(y) = v] \leq \frac{4}{m^2},$$

tedy systém je 2-nezávislý až na konstantu. $\square$

2.9 Sestrojte k -nezávislý systém hešovacích funkcí ze $\mathbb{Z}_p$ do $[m]$. Zdůvodněte k -nezávislost (můžete využít lemma o modulení).

Odpověď

Konstrukce. Necht p je prvočíslo, $m \leq p$ a $k \geq 1$. Uvažujme rodinu polynomiálních hešovacích funkcí

$$H = \{h_{\mathbf{a}} \mid \mathbf{a} = (a_0, \dots, a_{k-1}) \in \mathbb{Z}_p^k\}, \quad h_{\mathbf{a}}(x) = \sum_{i=0}^{k-1} a_i x^i \pmod{p}.$$

Pro zobrazení do $[m]$ použijeme

$$h(x) := h_{\mathbf{a}}(x) \bmod m.$$

Tvrzení. Rodina H je k -nezávislá (resp. $(k, O(1))$ -nezávislá po redukci modulo m).

Zdůvodnění. Pro libovolné různé body $x_1, \dots, x_k \in \mathbb{Z}_p$ a libovolné hodnoty $y_1, \dots, y_k \in \mathbb{Z}_p$ existuje právě jeden polynom stupně nejvýše $k - 1$, který splňuje $h(x_i) = y_i$ pro všechna i (Lagrangeova interpolace). Proto je zobrazení na $\mathbb{Z}_p$ přesně k -nezávislé. Po redukci modulo m se pravděpodobnosti mohou změnit jen o konstantní faktor (lemma o modulu), takže k -nezávislost zůstává zachována až na konstantu.

2.10 Sestrojte 2-nezávislý systém hešující řetězce délky nejvýše L nad abecedou $[a]$ do $[m]$ založený na polynomech („rolling hash“). Popište výhodu použití tohoto systému oproti jiným hešovacím funkcím.

Odpověď

Konstrukce (rolling hash). Nechť p je prvočíslo, $p > a^L$, a $m \leq p$. Zvolíme náhodně $b \in \mathbb{Z}_p$. Řetězec $s = s_0 s_1 \dots s_{\ell-1}$ ($\ell \leq L$) hešujeme jako

$$h(s) = \sum_{i=0}^{\ell-1} s_i b^i \pmod{p} \pmod{m}.$$

Tvrzení. Tento systém je 2-nezávislý (a tedy univerzální).

Zdůvodnění. Pro dva různé řetězce $s \neq t$ odpovídá rovnost $h(s) = h(t)$ nenulovému polynomu v proměnné b stupně nejvýše $L - 1$. Takový polynom má v tělese $\mathbb{Z}_p$ nejvýše $L - 1$ kořenů, tedy

$$\Pr[h(s) = h(t)] \leq \frac{L - 1}{p}.$$

Pro dostatečně velké p je tedy kolizní pravděpodobnost $O(1/p)$, což dává 2-nezávislost (resp. univerzalitu).

Zesílení na 2-nezávislost. Výše jsme ukázali, že polynomiální (rolling) hash je d -univerzální s $d = L - 1$, tj. pro libovolné různé řetězce $s \neq t$ platí

$$\Pr[h(s) = h(t)] \leq \frac{L - 1}{p}.$$

Abychom získali skutečnou 2-nezávislost pro zobrazení do $[m]$, použijeme standardní kompozici s lineární hešovací funkcí

$$g(x) = (ax + b) \pmod{p} \pmod{m},$$

kde $a, b \in \mathbb{Z}_p$ jsou zvoleny náhodně. Pro dostatečně velké $p = \Omega(Lm)$ je výsledná složená funkce $(2, O(1))$ -nezávislá (resp. 2-nezávislá až na konstantu).

Výhoda rolling hashe. Hlavní výhodou rolling hashe je *inkrementální výpočet*: hash podřetězce lze aktualizovat z hashe předchozího podřetězce v konstantním čase. To je klíčové pro algoritmy nad řetězci (Rabin–Karp, porovnávání podřetězců, LCP apod.).

2.11 Popište a analyzujte Bloomův filtr. Uveďte příklad jeho praktického použití.

Odpověď

Definice. Bloomův filtr je pravděpodobnostní datová struktura pro reprezentaci množiny, která podporuje dotazy typu „*je prvek v množině?*“. Nikdy nevrací falešně negativní odpověď (prvek, který byl vložen, je vždy rozpoznán), ale může vrátit *falešně pozitivní* odpovědi (prvek nebyl vložen, ale filtr tvrdí, že ano).

Konstrukce (intuice). Bloomův filtr lze chápat jako bitové pole, do něhož se prvky „rozprostrou“ pomocí více hešovacích funkcí. Konkrétně se skládá z:

- bitového pole $B[0..m-1]$ inicializovaného nulami,
- k nezávislých hešovacích funkcí $h_1, \dots, h_k : U \rightarrow [m]$.

Při vložení prvku x nastavíme bity $B[h_1(x)], \dots, B[h_k(x)]$ na 1. Každý prvek tedy obsadí k různých pozic v poli.

Dotaz (intuice). Pro dotaz na prvek x použijeme *stejně hešovací funkce*, které by byly použity při jeho vložení. Konkrétně zkontrolujeme bity

$$B[h_1(x)], \dots, B[h_k(x)].$$

- Pokud je alespoň jeden z těchto bitů roven 0, pak x v množině *určitě není*, protože při vložení by tento bit musel být nastaven.
- Pokud jsou všechny tyto bity rovny 1, odpovíme, že x je v množině *pravděpodobně*.

Falešná pozitivita nastává tehdy, když byly všechny kontrolované bity nastaveny jinými prvky.

Vícepásmová intuice. Bloomův filtr lze chápat jako k „pásem“ nad jedním bitovým polem: každý prvek musí projít všemi pásmy, aby byl považován za přítomný. Čím více pásem (k), tím menší je šance, že náhodný prvek projde všemi, ale zároveň se rychleji zaplňuje bitové pole.

Pravděpodobnost falešné positivity (intuice). Po vložení n prvků je pravděpodobnost, že konkrétní bit zůstane nulový, přibližně $e^{-kn/m}$. Pravděpodobnost falešné positivity je tedy přibližně

$$\left(1 - e^{-kn/m}\right)^k,$$

protože dotazovaný prvek musí „projít“ všemi k pásmy současně. Optimální volba

$$k = \frac{m}{n} \ln 2$$

vyvažuje počet pásem a zaplnění bitového pole a minimalizuje pravděpodobnost falešné positivity.

Vlastnosti.

- velmi malá paměťová náročnost,
- konstantní čas dotazu i vložení,
- neumožňuje mazání prvků (bez rozšíření, např. counting Bloom filter).

Praktické použití. Bloomův filtr se používá jako *rychlý filtr*: nejprve levně vyloučí prvky, které v množině nejsou, a teprve poté se provede dražší operace. Typické použití:

- databáze a cache (vyloučení neexistujících klíčů),
- distribuované systémy (Bigtable, Cassandra),
- síťové aplikace a webové crawly.

2.12 Definujte k-d stromy a ukažte, že 2D intervalové dotazy trvají $\Omega(\sqrt{n})$.

Odpověď

Definice k-d stromu. k-d strom je binární strom pro body v $\mathbb{R}^k$, který střídavě dělí prostor podle jednotlivých souřadnic. Na úrovni i se body dělí podle souřadnice $i \bmod k$, typicky mediánem.

Dotazy. Podporuje dotazy:

- bodové vyhledávání,
- ortogonální intervalové dotazy,
- dotazy na nejbližšího souseda.

Dolní mez pro 2D intervalové dotazy. V rovině ($k = 2$) existuje množina n bodů a obdélníkový dotaz, který protíná $\Omega(\sqrt{n})$ regionů stromu, aniž by obsahoval mnoho bodů.

Idea důkazu. k -d strom rozděluje rovinu na $O(n)$ obdélníků. Lze zkonstruovat dotazový obdélník, který:

- protíná mnoho těchto obdélníků,
- neobsahuje jejich celé podstromy.

Takový dotaz musí navštívit $\Omega(\sqrt{n})$ uzlů stromu, i když vrátí jen málo bodů.

Důsledek. Nejhorší případ pro 2D intervalové dotazy v k -d stromech má složitost

$$\Omega(\sqrt{n}),$$

což je výrazně horší než u intervalových stromů.

2.13 Ukažte, jak dynamizovat dvou dimenzionální intervalové stromy (tedy Range trees), stačí Insert.

Odpověď

Připomenutí struktury. Dvourozměrný intervalový strom se skládá:

- z *primárního* vyhledávacího stromu podle souřadnice x ,
- v každém uzlu v ze *sekundární struktury* nad množinou bodů P_v , seřazené podle souřadnice y .

Použité vyvažování. Primární strom i všechny sekundární struktury jsou implementovány jako vyhledávací stromy s *líným vyvažováním* (např. BB[α]-stromy). Vyvažování se tedy neprovádí po každé operaci, ale pouze při výrazném porušení rovnováhy, a jeho cena se analyzuje amortizovaně.

Operace INSERT(p). Nechť $p = (x_p, y_p)$ je vkládaný bod.

1. V primárním stromu vložíme bod p podle souřadnice x_p .
2. Při návratu po cestě ke kořeni vložíme bod p do sekundární struktury každého navštíveného uzlu.

Amortizovaná analýza.

- Cesta v primárním stromu má délku $O(\log n)$.
- Vložení do jedné sekundární struktury stojí $O(\log n)$ amortizovaně.

- Přestavby podstromů nastávají až po $\Omega(k)$ operacích a jejich cena se amortizuje na konstantu na operaci.

Celková složitost. Amortizovaná časová složitost operace INSERT je:

$$O(\log^2 n).$$

Poznámka. Frakční kaskádování zrychluje pouze dotazy. Pro dynamické operace (Insert/Delete) zůstává složitost $O(\log^2 n)$.

2.14 Ukažte, jak použít suffixové pole a LCP pole na nalezení nejdelšího společného podřetězce dvou řetězců.

Odpověď

Zadání. Máme dva řetězce A a B a chceme nalézt jejich nejdelší společný podřetězec (LCS).

Konstrukce.

1. Sestavíme řetězec

$$S = A\#B\$,$$

kde $\#$, $\$$ jsou znaky nepatřící do abecedy.

2. Spočítáme suffixové pole SA a LCP pole pro S .

Pozorování. Nejdelší společný podřetězec odpovídá nejdelšímu společnému prefixu dvou sufixů, z nichž jeden pochází z A a druhý z B . Takové dva sufixy se v suffixovém poli nutně nacházejí vedle sebe.

Algoritmus. Projdeme pole LCP a hledáme maximum $LCP[i]$ takové, že sufixy $SA[i]$ a $SA[i - 1]$ pocházejí z různých řetězců.

Složitost. Celková časová složitost je:

$$O(|A| + |B|).$$

2.15 Ukažte, jak paralelizovat (a,b)-strom.

Odpověď

Základní pozorování. Operace FIND, INSERT a DELETE v (a,b)-stromu procházejí vždy pouze jednu cestu od kořene k listu a výška stromu je $O(\log n)$. To umožňuje, aby operace probíhající v různých podstromech běžely paralelně.

Preventivní úpravy při sestupu. Klíčovou myšlenkou paralelizace je, že strukturální změny (štěpení a slučování) se neprovádějí až při návratu nahoru, ale *preventivně již při sestupu stromem*.

- Při INSERT: pokud při sestupu narazíme na uzel, který má maximální povolený počet klíčů ($b - 1$), uzel ihned rozdělíme.
- Při DELETE: pokud při sestupu narazíme na uzel s minimálním počtem klíčů ($a - 1$), ihned si klíče vypůjčíme ze sourozence nebo provedeme sloučení.

Díky tomu se po dosažení listu už nemusí provádět žádné strukturální změny směrem nahoru.

Role podmínky $b \geq 2a$. Podmínka $b \geq 2a$ zajišťuje, že při preventivním štěpení nebo slučování:

- nikdy nevznikne podměrečný uzel,
- oba vzniklé uzly splňují omezení (a,b)-stromu.

To je klíčové pro korektnost algoritmu bez nutnosti globální synchronizace.

Paralelní přístup ke stromu. Při paralelním provádění operací:

- každý proces uzamyká pouze uzly na své cestě,
- zámky se uvolňují postupně při sestupu,
- operace v různých větvích stromu se neblokují.

Strukturální změny jsou lokální a týkají se pouze konstantního počtu uzlů.

Důsledky.

- FIND je vysoce paralelní operace,
- INSERT a DELETE vyžadují pouze lokální synchronizaci,
- strom zůstává vždy vyvážený a splňuje podmínky (a,b)-stromu.

2.16 Navrhněte a analyzujte bezzámkovou implementaci zásobníku.

Odpověď

Bezzámkový zásobník (lock-free stack). Zásobník je reprezentován jednosměrným spojovým seznamem. Ukazatel `top` atomicky ukazuje na vrchol

zásobníku. Cílem je implementovat operace PUSH a POP bez použití zámků, pouze pomocí atomických operací.

Atomické primitivum CAS. Základním nástrojem je operace

`CAS(addr, old, new),`

která atomicky porovná obsah `*addr` s hodnotou `old` a pokud jsou shodné, zapíše `new`. V opačném případě neprovede žádnou změnu.

Operace Push.

1. Vytvoříme nový uzel a nastavíme `new->next := top`.
2. Pomocí CAS se pokusíme změnit `top` z původní hodnoty na `new`.
3. Pokud CAS selže (mezitím někdo změnil `top`), operaci opakujeme.

Operace Pop.

1. Načteme aktuální hodnotu `top`.
2. Pokud je zásobník prázdný, skončíme.
3. Jinak se pomocí CAS pokusíme změnit `top` na `top->next`.
4. Při neúspěchu CAS operaci opakujeme.

Analýza vlastností.

- *Lock-free*: v každém okamžiku alespoň jedno vlákno postupuje.
- Neexistují deadlocky ani priority inversion.
- Operace mohou opakovaně selhávat (busy-waiting), ale celkový postup systému je zaručen.

2.17 Popište atomická primitiva a jejich vlastnosti. Vysvětlete problém ABA a jeho řešení.

Odpověď

Atomická primitiva. Atomická primitiva jsou operace, které se vykonávají nedělitelně z pohledu ostatních vláken. Typická atomická primitiva jsou:

- Read/Write atomické proměnné,
- Fetch-and-Add,
- Test-and-Set,

- **Compare-and-Swap (CAS).**

Z nich je CAS nejpoužívanější pro návrh bezzámkových struktur.

Vlastnosti CAS. CAS umožňuje podmíněnou atomickou změnu paměti a je dostatečně silné k implementaci libovolné bezzámkové datové struktury.

Problém ABA. Při použití CAS může nastat situace:

$$A \rightarrow B \rightarrow A,$$

kdy se hodnota sledované proměnné mezitím změnila, ale vrátila se na původní hodnotu. CAS tuto změnu nerozpozná, přestože se struktura dat mezitím zásadně změnila (např. uzel byl odebrán, uvolněn a znovu použit).

Důsledky. Problém ABA může vést k porušení korektnosti bezzámkové datové struktury, zejména při práci s dynamickou pamětí.

Řešení problému ABA. Používají se zejména tyto techniky:

- **Tagované ukazatele:** spolu s adresou se porovnává i čítač změn.
- **Hazard pointers:** vlákna explicitně oznamují, které uzly právě používají, takže tyto uzly nelze uvolnit.
- **Epoch-based reclamation:** paměť se uvolňuje opožděně až po skončení všech vláken v dané epoše.

Shrnutí. Atomická primitiva umožňují efektivní paralelní algoritmy bez zámků, ale vyžadují pečlivé řešení správy paměti a problému ABA.